

Obidi Jugu: An Interactive Application for Weighted TSP on a Sparse Road Network

Irfan Hadžić, Volodymyr Rodin, Kemal Sivro

Faculty of Electrical Engineering, University of Sarajevo
Artificial Intelligence, instructor Izudin Džafić, academic year 2025/26

Abstract

Obidi Jugu is an interactive desktop application for the Travelling Salesman Problem (TSP) on an editable road graph of the former Yugoslav region. It optimises over a Dijkstra metric closure of a sparse graph and uses the transition cost $c(u, v) = d(u, v)w(v)$, making the objective asymmetric when destination weights differ. This report presents the problem model, Nearest Neighbor (NN), best-improvement 2-opt, Simulated Annealing (SA), a Genetic Algorithm (GA), and exact branch-and-bound. The application automatically selects an exact benchmark for at most ten required cities and a 100-cycle NN+2-opt reference for larger instances. Its interface supports instance editing, solution replay, route tables, and comparison of weighted cost against the selected reference. Background computation and resize rendering optimisations keep the interface responsive. The report describes the implemented application and its mathematical model; it does not claim a measured ranking of algorithm performance.

Keywords: travelling salesman problem, sparse road graph, metric closure, 2-opt, simulated annealing, genetic algorithm, exact optimisation, algorithm benchmarking

1 Introduction

The Travelling Salesman Problem asks for a minimum-cost Hamiltonian cycle over a required set of vertices and is NP-hard [1]. Educational demonstrations often use a complete Euclidean graph, but transport networks are sparse and a visually direct edge need not be a legal road. Obidi Jugu therefore separates the road graph from the optimisation graph: shortest road paths define the distances between required cities, while Open cities can act as transit vertices without becoming mandatory tour stops.

The application lets a user construct, solve, inspect, and replay weighted TSP instances. Its purpose is to connect the mathematical tour order to the underlying road network and to make the effects of city roles, weights, and algorithm parameters visible.

The main contributions are:

- a weighted asymmetric TSP formulation over the Dijkstra closure of an editable road network;
- implementations of NN, 2-opt, SA, GA, and exact branch-and-bound in one interactive application;
- an automatic in-application benchmark with a clearly identified exact or heuristic reference; and
- an interface that connects solutions to expanded road paths while keeping long computations cancellable.

2 Problem Formulation

2.1 Sparse road graph and roles

Let $G = (V, E)$ be an undirected sparse road graph. Each city has one of four roles:

- **Start:** the unique tour anchor, fixed at permutation index zero;
- **Goal:** a mandatory tour vertex;

- **Open**: an optional shortest-path transit vertex; and
- **Blocked**: excluded from both routes and tours.

The required set is $R = \{s\} \cup V_{\text{Goal}}$. A valid tour is a permutation $\pi = (s, \pi_2, \dots, \pi_{|R|})$ containing every member of R exactly once.

2.2 Metric closure

For each passable source, Dijkstra's algorithm [2] computes the shortest road distance

$$d(u, v) = \min_{p: u \rightsquigarrow v} \sum_{(a,b) \in p} \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}. \quad (1)$$

The all-pairs values form a metric closure over the passable vertices. Optimisation can therefore treat required cities as a complete graph, while visualization expands each selected hop back into the corresponding road path. If any required pair is unreachable, no finite tour exists.

2.3 Destination-weighted objective

Each city v has weight $w(v) > 0$. Entering it costs

$$c(u, v) = d(u, v)w(v), \quad (2)$$

and the closed tour objective is

$$C(\pi) = \sum_{i=1}^{|R|} c(\pi_i, \pi_{(i \bmod |R|)+1}). \quad (3)$$

Although $d(u, v) = d(v, u)$, generally $c(u, v) \neq c(v, u)$. The implementation consequently evaluates the complete candidate after a reversal instead of using the constant-time symmetric 2-opt delta formula. The application also reports unweighted road distance, but Equation 3 is the optimised objective.

3 Algorithms

3.1 Nearest Neighbor and best-improvement 2-opt

NN starts at s and repeatedly appends the unvisited required city with minimum transition cost. Construction requires $O(n^2)$ comparisons for $n = |R|$. The optional 2-opt phase examines every non-adjacent reversal that leaves s fixed, applies the best strict improvement, and repeats for at most the configured number of cycles [3].

Algorithm 1 Nearest Neighbor followed by best-improvement 2-opt

```

1:  $\pi \leftarrow [s], U \leftarrow R \setminus \{s\}$ 
2: while  $U \neq \emptyset$  do
3:    $v \leftarrow \arg \min_{u \in U} c(\pi_{\text{last}}, u)$ 
4:   append  $v$  to  $\pi$ ; remove  $v$  from  $U$ 
5: end while
6: for  $k = 1, \dots, K$  do
7:   evaluate every admissible segment reversal of  $\pi$ 
8:   if no reversal strictly reduces  $C(\pi)$  then
9:     break
10:  end if
11:  apply the best reversal
12: end for
13: return  $\pi$ 

```

One best-improvement cycle is $O(n^3)$ here: $O(n^2)$ candidate reversals, each fully evaluated in $O(n)$ because the cost is asymmetric.

3.2 Simulated Annealing

SA begins with either a random permutation or the NN tour. One call to `step()` proposes one random 2-opt reversal. An improving candidate is always accepted; a non-improving candidate with $\Delta = C(\pi') - C(\pi) \geq 0$ is accepted with probability [4, 5]

$$P(\text{accept}) = \exp(-\Delta/T). \quad (4)$$

After L valid trials, $T \leftarrow \alpha T$. The current tour and best-so-far tour are stored independently, and the Start city never moves.

3.3 Genetic Algorithm

The GA represents an individual as a Start-pinned permutation. Fitness is $1/C(\pi)$. The application supports roulette, tournament, and rank selection; it also provides Order Crossover, swap or inversion mutation, and elitist replacement. Population size, generation count, crossover rate, mutation rate, and elitism are configurable. Order Crossover preserves membership and relative order, so no repair stage is required. With population P , G generations, and $O(n)$ tour evaluation, the principal work is $O(PGn)$ [6, 7].

3.4 Exact reference

The application uses incremental branch-and-bound, seeded by an NN incumbent. Its lower bound sums an independent minimum outgoing transition for the current endpoint and every unvisited vertex. Completion certifies the optimum within floating-point arithmetic, but worst-case time remains factorial.

3.5 Automatic benchmark and delta

Every Solve or Resolve run also computes a reference using the same weighted objective. For $n = |R| \leq 10$, the reference is obtained by exact branch-and-bound. For $n > 10$, it is NN followed by at most 100 best-improvement 2-opt cycles. The threshold counts the Start and Goal cities rather than every displayed transit vertex.

The information card identifies the reference as exact or heuristic and reports the absolute cost difference

$$\Delta C = C_{\text{solution}} - C_{\text{reference}}. \quad (5)$$

When the reference is exact, this is the difference from a proven optimum within floating-point accuracy. When the reference is NN+2-opt, a negative delta means that the selected algorithm improved on that heuristic; the reference is not labelled as a true minimum.

A separate unrestricted Exact TSP option is available for demonstrating the computational cost of exact search. It runs in the background and can be cancelled using Reset. Its completion time is not bounded by the automatic ten-city threshold.

4 Application and Architecture

The Windows C++ application uses natID and GTK 4. `DataRepository` owns cities, roads, JSON loading, and the metric closure. `MainView` coordinates a `SidePanelView`, `MapView`, the active polymorphic solver, timers, and optional worker jobs. Solver state is never drawn from a worker thread: the GUI polls a completion flag and then transfers the completed tour.

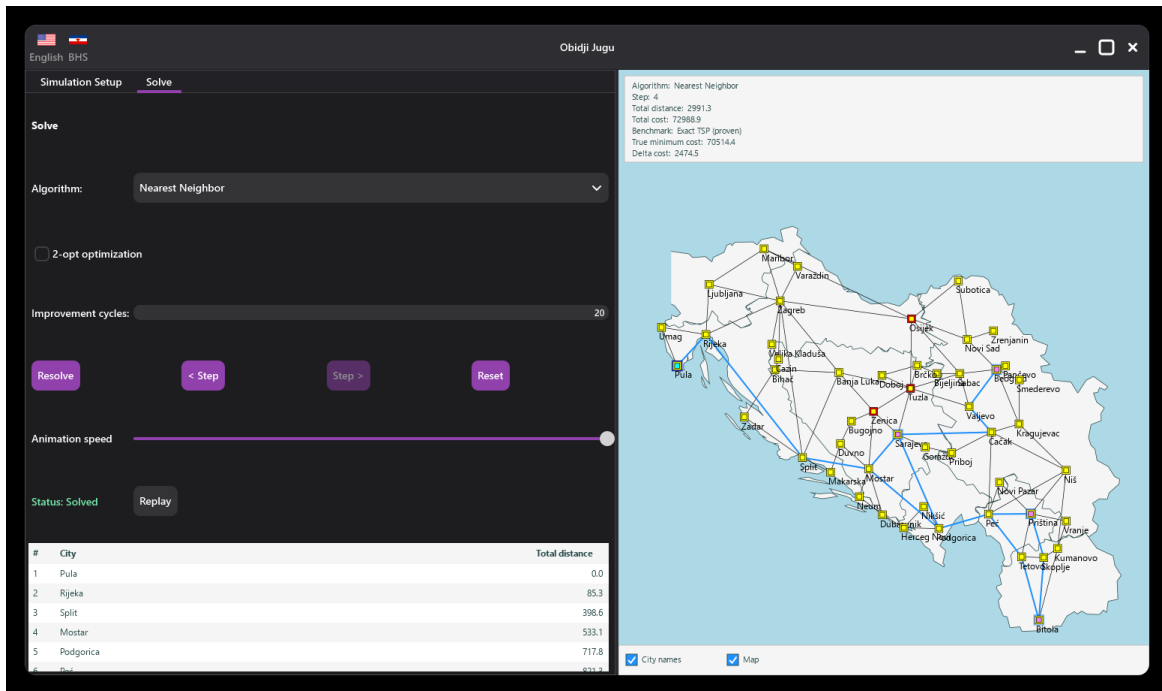


Figure 1: Completed NN solution. The map expands required-city hops into road segments; the panel reports weighted cost, geographic distance, exact benchmark, delta, status, and the expanded route table.

The side panel separates instance editing from solving. Users can change roles and weights, toggle direct roads on the map, select an algorithm, step forward or backward where snapshots are available, solve on a background thread, cancel with Reset, and replay a finished route without re-solving. Static map geometry is cached during resizing, labels are temporarily simplified, and the constrained splitter prevents the map or controls from collapsing.

4.1 Editing, replay, and visual controls

The Simulation Setup tab contains city creation and editing, role and weight selection, connection editing, randomisation controls, and a city table whose selection is linked to the editor. The Solve tab contains algorithm parameters, Solve/Resolve, Reset, animation speed, Replay, and the expanded route table. Solve/Resolve starts a fresh computation; Replay animates the completed route without recalculating it. The status label distinguishes solved and unsolved states, and Replay is disabled until a valid result exists.

The route table shows cities in expanded travel order and cumulative unweighted road distance. A transit city can therefore appear more than once even though each required city appears once in the TSP permutation. Total weighted cost remains a separate quantity. The map's City names and Map checkboxes reduce label and polygon clutter. English and Bosnian interface translations are supplied through XML resource catalogs.

4.2 Rendering and execution optimisations

During active resizing, redraw frequency is limited and expensive details such as city labels are temporarily suppressed. Cached static map imagery is reused, followed by a complete redraw when resizing settles. Simplified coastline and border geometry reduces the number of points processed. Dynamic city markers, connections, and solution overlays must reflect the current state rather than being retained in a stale background image.

Solver and benchmark work runs on a background thread using a private repository copy. The GUI polls for completion and displays the finished solution. Reset requests cancellation and prevents an obsolete result from replacing the current state. Route playback uses incremental GUI timer callbacks;

the animation is not a second solver computation or a parallel rendering operation.

5 Discussion

The algorithms offer different search strategies. NN makes greedy weighted-cost decisions. Best-improvement 2-opt accepts only cost-reducing reversals, while SA can accept a worse current tour to explore other regions of the search space. GA explores a population through selection, crossover, mutation, and elitism. These differences explain why parameter choices and instance structure can lead to different final routes; they do not establish that one algorithm always outperforms another.

The in-application reference helps interpret a particular result. An exact reference establishes the minimum for that instance. A heuristic reference supplies a consistent comparison value for larger problems without proving optimality. Re-running SA or GA can produce a different route, but repeated identical results are also possible, especially on small required sets.

Destination weights distinguish the model from distance-only TSP. Increasing a destination weight changes the relative cost of entering that city from different predecessors. Consequently, minimising weighted cost need not minimise total road distance. The separate distance and cost displays make this modelling choice explicit.

6 Limitations

Exact branch-and-bound has factorial worst-case search complexity. The automatic threshold limits its use in routine comparisons, while the unrestricted option remains available for demonstrations. The NN+2-opt reference above ten required cities is a heuristic and cannot certify an optimality gap.

City weights are user-defined model coefficients rather than calibrated measurements of traffic. Interpreting them as congestion is illustrative: the current model has no time-dependent traffic, vehicle capacities, or multiple-vehicle routing. Distances are derived from the graph geometry and should not be interpreted as measured real-world driving distances without appropriate geographic calibration.

Solution quality depends on the graph, required cities, weights, and algorithm settings. This report contains no systematic performance measurements, so it makes no numerical claims about speed, success rates, or relative algorithm ranking.

7 Conclusion

Obidi Jugu combines an editable sparse road graph, asymmetric destination-weighted tour cost, configurable TSP algorithms, road-path expansion, solution replay, and cancellable background computation. Its automatic exact-or-heuristic reference and cost delta support direct inspection of individual solutions. Responsive rendering, independent tables, visual-clutter controls, and XML localisation complement the mathematical model and make the application suitable for interactive demonstrations of routing and optimisation.

References

- [1] D. L. Applegate, R. E. Bixby, V. Chvátal, and W. J. Cook, *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.
- [2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [3] G. A. Croes, “A method for solving traveling-salesman problems,” *Operations Research*, vol. 6, no. 6, pp. 791–812, 1958.

- [4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [5] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, “Equation of state calculations by fast computing machines,” *Journal of Chemical Physics*, vol. 21, no. 6, pp. 1087–1092, 1953.
- [6] J. H. Holland, *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [7] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.
- [8] OpenStreetMap contributors, “OpenStreetMap,” <https://www.openstreetmap.org/>, accessed September 2026.
- [9] I. Džafić, “natID: native interface design framework,” <https://github.com/idzafic/natID>, accessed September 2026.